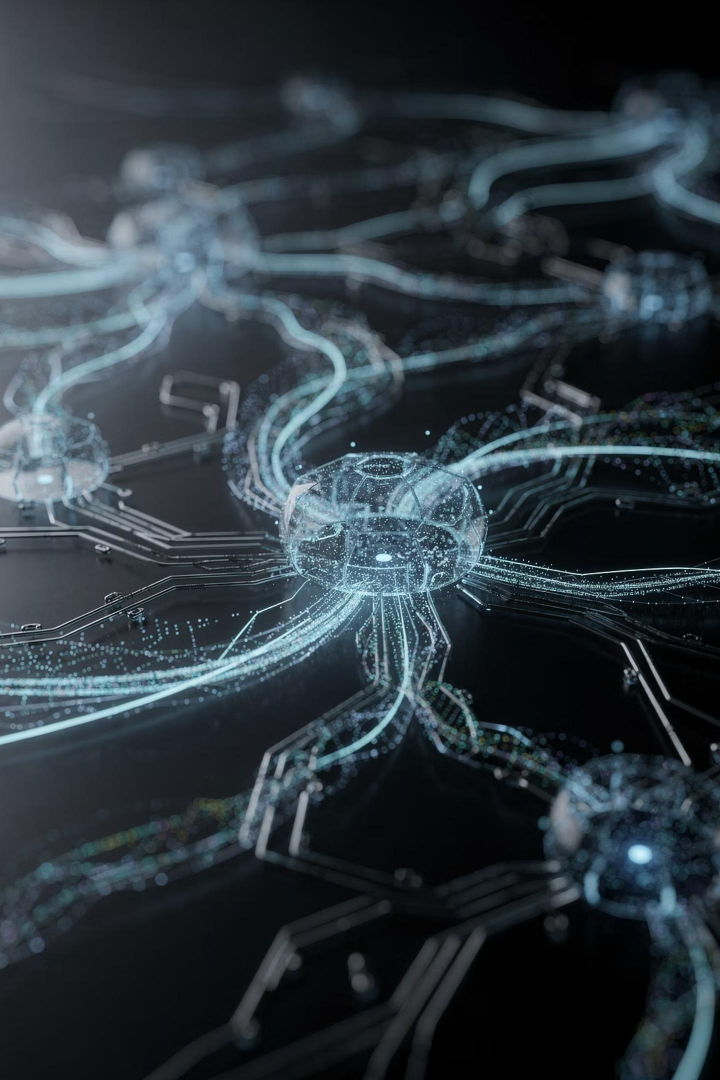




Big Data и искусственный интеллект

Семинар

Ведущий: Еникеев Марат

Что уже рассмотрели на лекции

На предыдущей лекции мы заложили теоретическую базу, необходимую для перехода к практике. Сегодня опираемся на эту основу и идём в практику.

Базовые понятия ML/LLM

Архитектура языковых моделей, принципы обучения, трансформеры и механизм внимания.

Роль и качество данных

Почему данные — ключевой актив ИИ-системы, и как их качество определяет результат.

Риски и ограничения ИИ

Галлюцинации, смещение данных, интерпретируемость и регуляторные ограничения.

Применение в банковском контуре

Конкретные примеры внедрения ИИ в финансовом секторе: скоринг, фрод, поддержка клиентов.

План сегодняшнего семинара

📝 План рабочий и может уточняться по ходу. Вопросы фиксируем по ходу — разбор в конце каждого блока.

1

Блок 1

LLM и системный подход

Архитектура, контекстное окно,
системные промпты, цепочки вызовов.

2

Блок 2

RAG и контекст-инжиниринг

Retrieval-Augmented Generation: принципы,
типовые ошибки, лучшие практики.

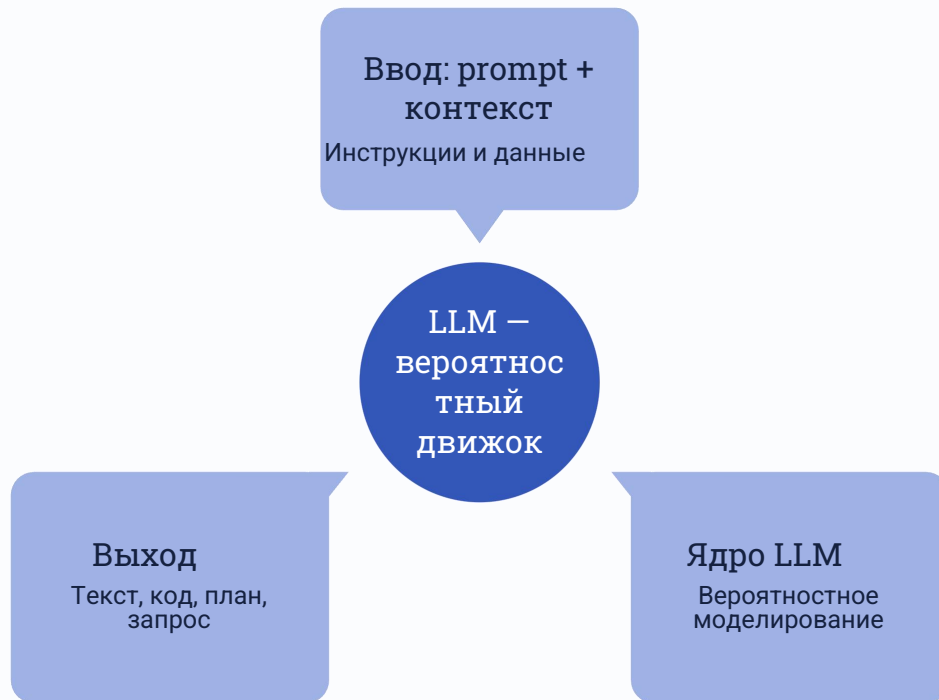
3

Блок 3

Практический кейс

Разбор простой системы на реальных
данных

Что такое LLM



Архитектурные решения должны учитывать вероятностную природу модели и компенсировать её ограничения.

Что LLM умеют хорошо

LLM наиболее эффективны в **рутинных когнитивных задачах**, где требуется трансформация, объяснение или структурирование информации. Именно здесь



Генерация кода

Написание шаблонов, функций, тестов и конфигураций по описанию задачи.



Объяснение

Разбор сложных концепций, документирование кода, перевод технического в понятное.



Преобразование форматов

Конвертация структур данных, рефакторинг, перевод между форматами (CSV → JSON → SQL).



Анализ кода и логов

Поиск аномалий, разбор ошибок, code review, выявление паттернов в данных.



Ограничения LLM

LLM — мощный инструмент, но с системными ограничениями. Грамотная архитектура учитывает их заранее и строит компенсирующие механизмы. Без этого модель остаётся полезным ассистентом, но не надёжным компонентом системы.

⚠ Ограничение контекста

Модель видит только то, что помещается в контекстное окно.
Длинные задачи требуют явного управления контекстом.

⚠ Нет свежих данных

Знания модели ограничены датой обучения. Для актуальной информации нужен внешний поиск или RAG.

⚠ Галлюцинации

Модель может уверенно генерировать неверные факты.
Критические ответы требуют верификации по источнику.

⚠ Дрейф на длинных задачах

На длинных цепочках рассуждений модель может «забывать» изначальную цель и отклоняться от задания.

⚠ LLM полезен как ассистент, но требует проверки и архитектурных ограничителей для использования в продакшене.

Почему одной модели уже недостаточно

Простая схема

prompt → модель → ответ

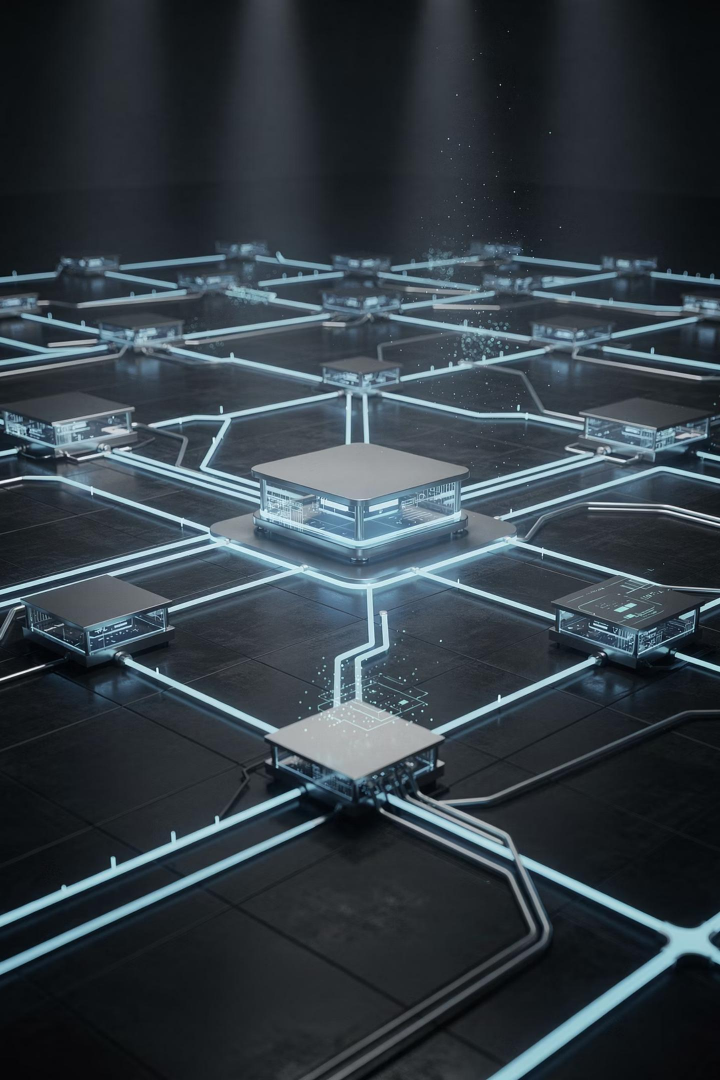
Только базовые ответы. Нет памяти, нет инструментов, нет контроля качества. Годится для экспериментов, но не для рабочих сценариев.

Рабочий контур

- **Контекст:** актуальные данные под задачу
- **Инструменты:** API, поиск, выполнение действий
- **Память:** история диалога и состояние
- **Валидация и контроль качества**

Модель в центре, окружённая системой поддержки.

 Ключевой сдвиг: от единственного вызова модели к спроектированной системе с компонентами, обеспечивающими надёжность.



Варианты доступа к моделям

Важна не только модель, но и канал доступа к ней

Проприетарные API

GPT, Claude, Gemini

Открытые модели

Llama, Qwen, DeepSeek, Mistral

Локальный запуск

Ollama, LM Studio, vLLM

Облачный роутер

OpenRouter, Together, Fireworks, Groq

КАЧЕСТВО

ЦЕНА

ЗАДЕРЖКА

ПРИВАТНОСТЬ

ПОДДЕРЖКА ИНСТРУМЕНТОВ

Промпт-инжиниринг как структура задачи

Качественный промпт: **структурированное техническое задание**. Для личного использования достаточно свободной формы; для агентных систем промпт должен быть строгим, проверяемым и воспроизводимым.

Цель (Goal)

Что именно нужно получить на выходе.

Контекст (Context)

Релевантные данные, фон задачи, ограничения среды.

Ограничения (Constraints)

Что нельзя делать, форматы, объёмы, инструменты.

Формат ответа (Format)

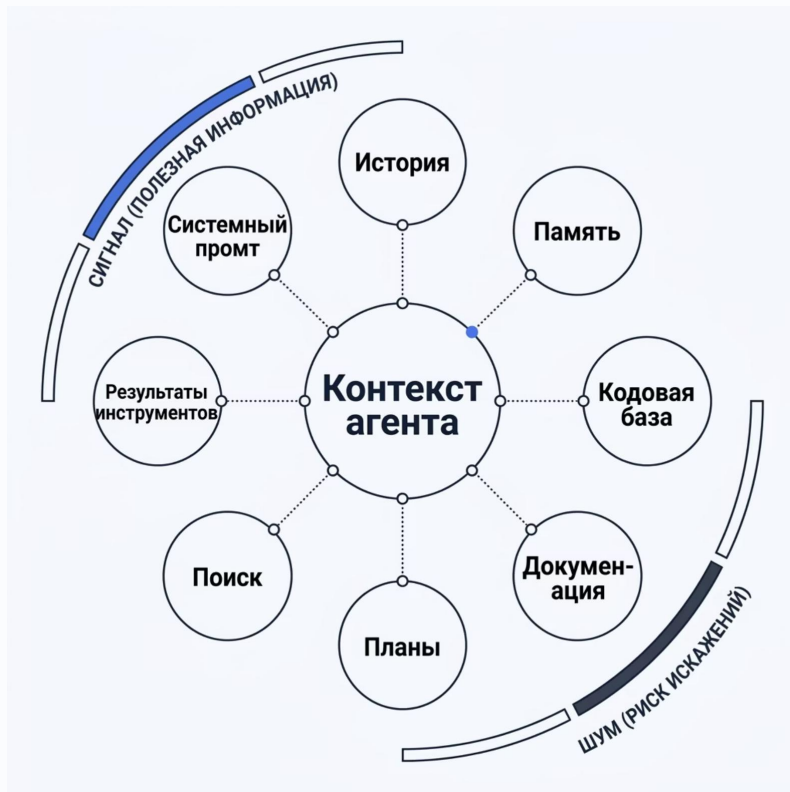
JSON, список, prose, код — явное указание структуры.

Критерий качества

Как определить, что ответ хороший и задача решена.

🔗 В личной практике — попросите модель сгенерировать промпт. В продакшене промпт нужно вручную доуточнять и валидировать на реальных кейсах.

Контекст-инжиниринг



Что входит в контекст агента

- System-промт — правила и роль
- История — предыдущие шаги диалога
- Память — накопленное состояние
- Кодовая база и документация
- Результаты инструментов и поиска



Signal vs Noise: важно отделять полезный сигнал от шума — лишний контекст снижает качество

Промпта недостаточно, важен корректный контекст

Что такое агент и что такое harness

Агент и harness — два разных слоя одной системы. Их разграничение критически важно для правильного проектирования ответственности и надёжности.

Модель Лишь инструмент для генерации текста	Harness Рабочая среда исполнения.
---	---



Статический vs динамический контекст

Статический контекст

- Системный промпт
- Правила и инструкции
- Определения инструментов
- MCP-схемы

Динамический контекст

- Добавляется по запросу
- Всплески объёма контекста
- Прогрессивная загрузка
- Рост стоимости токенов



Динамика напрямую влияет на token cost — следите за объёмом

Качество зависит от баланса: что держим статически, а что подгружаем динамически

Skills, Subagents и MCP: роли в системе

Три компонента выполняют разные, но **взаимодополняющие роли**. Ценность появляется в их связке.



Skills

Знания, playbook, чеклисты. Отвечают на вопрос **что и как** делать.



Subagents

Специализированные исполнители, делегирование и маршрутизация задач. Отвечают на вопрос **кто и когда**.



MCP

Инструменты, интеграции, внешние системы. Отвечают на вопрос **чем и с чем** работать.



Превращаем отдельные части в систему.

AI-first разработка: от «кодера» к оркестратору

От ручного набора кода пеход к проектированию и контролю системы. Человек остаётся в контуре принятия решений, но фокус его работы меняется принципиально.

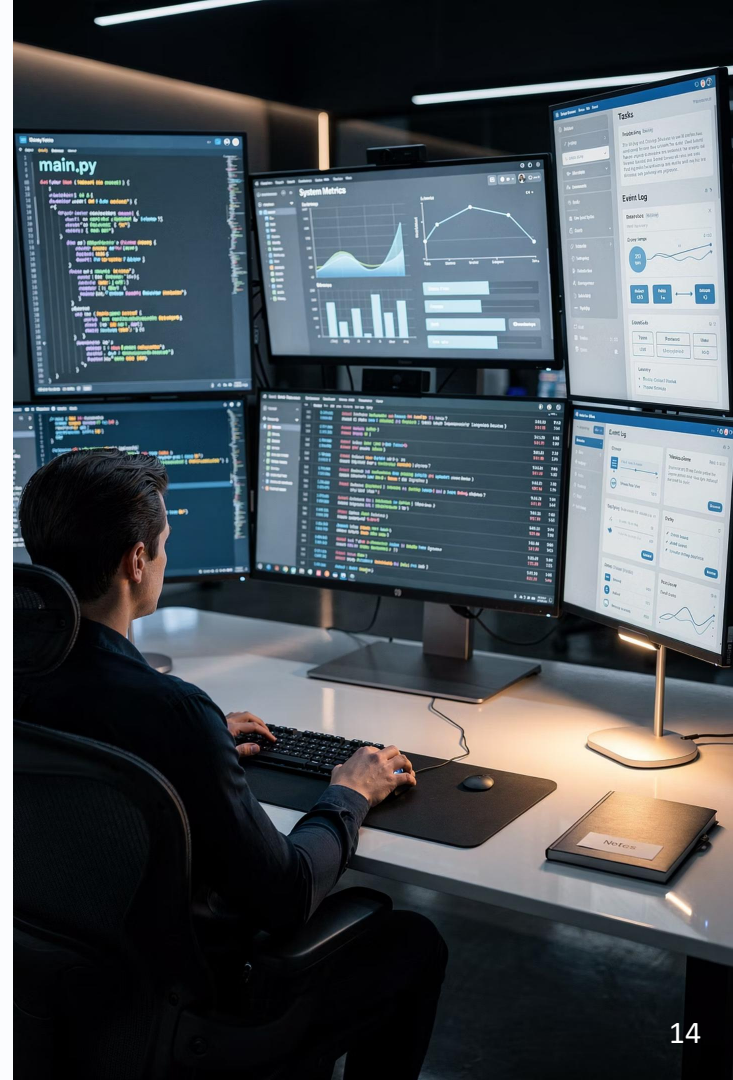
Code-only

- Фокус на ручном написании кода
- Изолированные задачи без системного контекста
- Ограниченный масштаб — скорость упирается в скорость набора
- ИИ используется точно, как подсказчик

+ LLM

- Меньше ручного кодинга, больше проектирования
- Верификация и контроль качества выходов
- ИИ встроен в основной workflow как компонент системы
- Человек управляет контекстом, архитектурой и решениями

❗ Важно: смещение роли не означает «человек не нужен». Акцент — на **human-in-control**: архитектурные решения и верификация остаются за инженером.



Эволюция AI-coding подходов

Зрелость AI-assisted разработки растёт по чёткой траектории — от ручных точечных запросов к оркестрированному циклу. На поздних этапах ключевая роль инженера — **контроль, верификация и управление контекстом**.



1. Copy-paste

Ручной поиск фрагментов кода, вставка без понимания контекста. Минимальная интеграция ИИ в процесс.



3. Context-driven

Работа через контекст репозитория, документов и данных. Модель понимает архитектуру проекта и принимает решения в её рамках.



2. Prompt-driven

Работа через запросы в чат. Быстрые ответы на конкретные вопросы, но без системного контекста репозитория.



4. Agent-native

Агенты выполняют части цикла разработки автономно: тесты, рефакторинг, code review. Инженер — в роли контролёра и верификатора.

Экосистема AI-инструментов

Выбирать по workflow, а не по хайпу

Консольные агенты

Repo, shell, тесты, diff — автономная работа с кодовой базой

IDE-агенты

Контекст кода, быстрые правки, локальная проверка

Прототипирование

Быстрое создание черновиков и интерфейсов

Спец-driven поток

Требования → дизайн → задачи → выполнение

- ❏ Один инструмент не закрывает все этапы разработки — важно различать режим ассистента и режим агента

Зачем нужен RAG

RAG (Retrieval-Augmented Generation) — архитектурный паттерн: сначала поиск релевантных фрагментов в базе знаний, затем генерация ответа на их основе. Формула: **RAG = поиск (retrieval) + генерация (generation)**.

Проблема «чистого» LLM

- Нет опоры на ваши документы и данные
- Риск галлюцинаций при фактических вопросах
- Сложно верифицировать источник ответа
- Знания ограничены датой обучения модели

Решение с RAG

- Сначала поиск релевантного контекста в индексе
- Ответ генерируется на основе найденных фактов
- Каждый ответ можно проверить по источнику
- База знаний обновляется независимо от модели

- ✓ RAG нужен для проверяемых ответов, а не для красивых формулировок. Ключевой критерий — возможность верификации по источнику.

Базовый pipeline RAG

RAG-пайплайн — это последовательность инженерных шагов. **Ошибка на раннем этапе переносится в финальный ответ:** некачественные чанки или неточный индекс не исправляются на этапе генерации.



Документы
Исходные тексты,
PDF, статьи, базы
знаний

Чанки
Разбивка на
смысловые
фрагменты

Эмбеддинги
Векторное
представление
каждого чанка

Индекс
Векторное
хранилище для
быстрого поиска

Топ-k поиск
Выбор наиболее релевантных фрагментов

Ответ
Генерация на основе найденного контекста

Чанкинг как рычаг качества

Границы чанков важнее, чем кажется. Размер и способ разбивки текста на фрагменты напрямую определяет, насколько точным будет retrieval — и, следовательно, финальный ответ.

✗ Слишком маленькие чанки

Фрагменты без достаточного контекста.

✗ Слишком большие чанки

Много нерелевантного шума в каждом фрагменте.

✓ Смысловые чанки

Разбивка по логическим единицам: абзацам, секциям, завершённым мыслям. Лучший баланс между контекстом и точностью поиска.

❗ Стратегии чанкинга: фиксированный размер с перекрытием, разбивка по параграфам, рекурсивный split, семантический чанкинг. Выбор зависит от типа документов.

Retrieval и top-k

Retrieval — сердце RAG-системы. Система возвращает **k наиболее семантически близких фрагментов** к запросу пользователя. Качество этого шага определяет всё остальное.

«Плохой top-k = хороший текст, но неправильный ответ»

#	Найденный фрагмент	Релевантность	Статус
1	«RAG — это архитектурный паттерн, где поиск предшествует генерации...»	0.94	✓ Используется
2	«Векторный индекс хранит эмбединги всех чанков документа...»	0.87	✓ Используется
3	«Чанки с перекрытием помогают сохранить контекст на границах...»	0.71	⚠ Проверить

Оптимальное значение k зависит от задачи. Используем малое k, то рискуем пропустить нужный фрагмент. Если слишком большое, то допускаем шум и увеличиваем стоимость. Reranking после retrieval улучшает точность финального набора.

System + Context: совместная работа

В RAG-системе два компонента работают в паре: **system-промпт задаёт правила**, а **контекст приносит факты**. Без правильного баланса система нестабильна.

System-промпт: правила ответа

- Отвечать только по предоставленному контексту
- Признавать нехватку данных явно
- Держать заданный формат результата
- Не генерировать факты из «общих знаний»

Context: факты для ответа

- Топ-k фрагментов из RAG-поиска
- Метаданные источников (для ссылок)
- Актуальные данные под конкретный запрос
- Структурированный, без лишнего шума

 System без контекста и контекст без system одинаково нестабильны. Только в паре они обеспечивают проверяемые, контролируемые ответы.

Типовые ошибки RAG

Устаревший индекс

Данные не обновляются — модель отвечает на основе неактуальной информации.

Шумные источники

Нерелевантные фрагменты попадают в контекст и снижают качество ответа.

Prompt injection

Вредоносные инструкции внутри документов могут перехватить поведение модели.

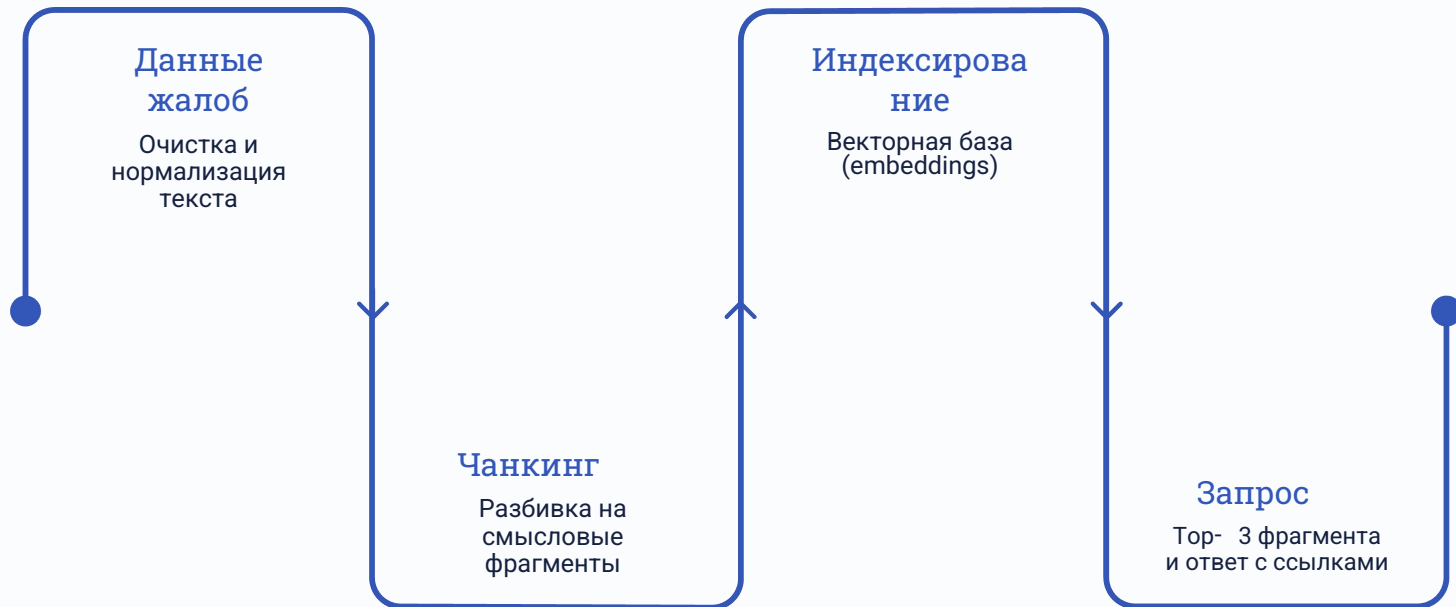
Нет оценки качества

Без метрик и eval-набора невозможно понять, работает ли система корректно.

RAG — это инженерный процесс, а не одна настройка. Системный подход, регулярное обновление и оценка качества — обязательные составляющие.

Мини-кейс: RAG на данных Kaggle

Разберём конкретный пример построения RAG-системы поверх данных жалоб клиентов банков: от входного csv-файла до верифицируемого ответа с источниками.



Ключевой принцип

Ответ должен быть **проверяемым по источнику**. Каждый ответ содержит ссылку на исходный документ — пользователь может верифицировать информацию самостоятельно.

Что даёт такой подход

- Снижение галлюцинаций за счёт заземления на реальный контент
- Прозрачность: каждый ответ трассируется до источника
- Возможность аудита и улучшения системы по реальным запросам

Домашнее задание

Свой учебный RAG на своих данных

Ваша задача — самостоятельно построить полный RAG-pipeline на собственных данных, используя образец из репозитория. Повторите каждый шаг: от загрузки данных до ответа с источником в Streamlit-интерфейсе.

ОБРАЗЕЦ

github.com/MaratNotes/rag-tutorial

01

Данные → Чанки

Подготовьте минимум 10 текстовых документов и разбейте их на фрагменты.

02

Индекс → Поиск

Постройте векторный индекс и реализуйте семантический поиск по запросу.

03

Ответ с источником → Streamlit

Генерируйте ответ с указанием источника и оберните всё в веб-интерфейс.

Что делать?

Откройте папку `homework/`, изучите план и двигайтесь итерациями 0–8.

Команды запуска

```
uv sync
uv run python scripts/build_index.py
uv run streamlit run app/main.py
```

Критерии сдачи

- Pull Request в репозиторий
- Файл `homework/SUBMISSION.md` со ссылкой на ваш репо

Минимальные требования

- 1к текстовых документов
- 3 вопроса с правильным ответом
- 1 вопрос с корректным отказом
- `pytest` — все тесты зелёные ✓

Спасибо за внимание!

Big Data и искусственный интеллект · Семинар

Сегодня мы прошли путь: **LLM** → **RAG** → **ваш проект на своих данных**

Домашнее задание

Полный RAG-pipeline в репозитории rag-tutorial. Двигайтесь итерациями, не бойтесь экспериментировать.

Сдача работы

Оформите Pull Request и добавьте файл homework/SUBMISSION.md со ссылкой на ваш репозиторий.

Вопросы и связь

Задавайте вопросы в чат семинара. Обратная связь и разбор ошибок — приветствуются.

Еникеев Марат · Telegram: t.me/marat_notes

Коротко о сложном: Инжиниринг данных, бэкенд, ИИ и личный опыт.